

The goal of this machine problem is to build an implementation of a list ADT which includes a set of fundamental procedures to manipulate the list. You will implement the list using two-way linked lists, and define a general-purpose interface that will allow us to use this list ADT for the remainder of the semester. Utilizing the basic list ADT, you will then re-write the MP1 honeypot application to utilize the new list ADT.

A key element in this assignment is to develop a well-designed abstraction for the list so that we can return to this interface and replace it with either (a) a different application that needs access to a list or (b) change the underlying mechanism for organizing information in a sorted list (e.g., other than a two-way linked list).

You are to write a C program that will maintain the **two** lists of packet information, one sorted and the other unsorted. The list ADT provides a general list package that is used for both lists. The code **must** consist of the following files:

lab2.c	– contains the main() function, menu code for handling simple input and output, and any other functions that are not part of the ADT.
ids_support.c	– contains subroutines that support handling of alert records.
l1list.c	– The two-way linked list ADT. The interface functions must be exactly defined as described below. You can include additional private functions but the interface cannot be changed. The functions must operate exactly as described in the comments.
ids_support.h	– The data structures for the specific format of the alert records, and the prototype definitions.
l1list.h	– The data structures and prototype definitions for the list ADT.
datatypes.h	– Key definitions of the snort alert structure
makefile	– Compiler commands for all code files

Part 1: The two-way linked list ADT

Your program must use a two-way linked list to implement the list ADT. A list is an object, and `l1list_construct()` is used to generate a new list. There can be any number of lists active at one time. For a specific list, there is no limit on the maximum size, and elements can be added to the list until the system runs out of memory. Your program must not have a memory leak, that is, at no time should a block of memory that you allocated be inaccessible.

The list ADT is designed so that it does not depend on the type of data that is stored except through a very limited interface. Namely, to allow the list ADT to be used with a specific instance of a data structure, we define the data structure in a header file called `datatypes.h`. This allows the compiler to map a specific definition of a structure to a type we call `data_t`. All of the procedures for the list ADT operate on `data_t`. In future programming assignments we can reuse the list ADT by simply modifying the `datatypes.h` file to change the definition of `data_t` and the comparison procedure prototype, and then recompile.

```
/* datatypes.h
 *
 * The data type that is stored in the list ADT is defined here. We define a
```

```

* single mapping that allows the list ADT to be defined in terms of a generic
* data_t.
*
* data_t: The type of data that we want to store in the list
*/
typedef struct alert_tag {
    int generator_id;    // ID of component generating alert
    int signature_id;    // ID of detection rule
    int revision_id;     // revision number of detection rule
    int dest_ip_addr;    // IP address of destination
    int src_ip_addr;     // IP address of source
    int dest_port_num;   // port number at destination
    int src_port_num;    // port number at source host
    int timestamp;       // time in seconds alert received
} alert_t;

/* the list ADT works on alert data of this type */
typedef alert_t data_t;

```

The list ADT must have the following interface, defined in the file `l1ist.h`.

```

/* l1ist.h
*
* You should not need to change anything in this file.  If you do, you
* must get permission from the instructor.
*/

#define LLPOSITION_FRONT -876543
#define LLPOSITION_BACK  -123456

typedef struct l1ist_element_tag {
    // private members for l1ist.c only
    data_t *data_ptr;
    struct l1ist_element_tag *ll_prev;
    struct l1ist_element_tag *ll_next;
} l1ist_elem_t;

typedef struct l1ist_header_tag {
    // private members for l1ist.c only
    l1ist_elem_t *ll_front;
    l1ist_elem_t *ll_back;
    int ll_entry_count;
    int ll_sorted_state;
    // Private method for l1ist.c only
    int (*compare_fun)(const data_t *, const data_t *);
} l1ist_t;

/* prototype definitions for functions in l1ist.c */
data_t * l1ist_access(l1ist_t *list_ptr, int pos_index);
l1ist_t * l1ist_construct(int (*fcomp)(const data_t *, const data_t *));
void l1ist_destruct(l1ist_t *list_ptr);
data_t * l1ist_elem_find(l1ist_t *list_ptr, data_t *elem_ptr, int *pos_index,
    int (*fcomp)(const data_t *, const data_t *));
int l1ist_entries(l1ist_t *list_ptr);
void l1ist_insert(l1ist_t *list_ptr, data_t *elem_ptr, int pos_index);
void l1ist_insert_sorted(l1ist_t *list_ptr, data_t *elem_ptr);
data_t * l1ist_remove(l1ist_t *list_ptr, int pos_index);

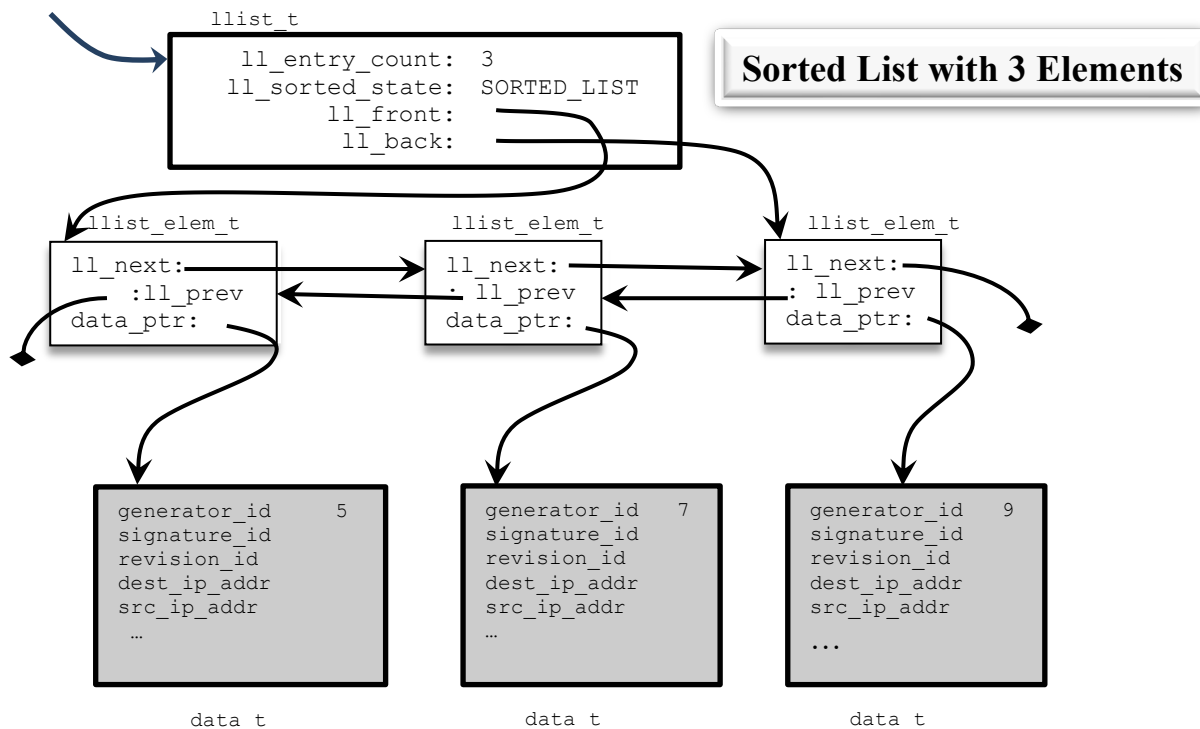
```

The details of the procedures for the list ADT are defined in the comment blocks of the `l1ist.c` template file. You can define additional *private* procedures to support the list ADT, but you must clearly document them as extensions.

When the list ADT is constructed, it must have the initial form as shown in the figure below.



The list ADT supports two variations on a list. A list can be maintained in either sorted or unsorted order, depending on the procedures that are utilized. Multiple lists can be concurrently maintained. Here is an example of the list ADT data structure when three `alert_t`'s have been added to the sorted list.



It is critical that the `l1ist.c` procedures be designed to not depend on the details of our alert records except through the definitions contained in `datatypes.h`. In particular, the letters “ids_”, and the member names in `alert_t` *must not* be found in `l1ist.c`. It is also critical that the internal details of the data structures in the `l1ist.c` file are kept private and are not accessed outside of the `l1ist.c` file, except by the provided `driver.c` for testing purposes. The members of the structures `l1ist_t` and `l1ist_elem_t` are private and their details *must not* be found in code outside of the `l1ist.c` file, except in the provided `driver.c`. In particular, the letters “->data_ptr”, “->ll_next”, “-

>ll_front", ">ll_entry_count", or any variation on ">ll_" are considered private to the list ADT and *must not* be found in any *.c file except llist.c or the provided driver.c.

Check that your code meets these requirements by using `make design`. See the makefile for comments. Test your code using `driver.c` (many changes will be needed to this file to test all your linked list functions). You can compile the driver with `make driver`.

Hint: Begin this assignment by completing all of Part 1 only. All work for Part 1 is contained in `llist.c`. Test your `llist.c` module using `driver.c`. Only when Part 1 is completed and tested should you move on to Part 2.

Part 2: The extended functions for alert records

Implement the following procedures in an `ids_support.c` file. The procedures should be implemented using the list ADT as the mechanism to store the records. The following header file defines the example prototype definitions. You may add additional helper prototypes in `ids_support.h`, but you cannot change the provided prototypes or the `llist.h` interface.

```
/* ids_support.h */

/* prototype function definitions */
/* function to compare packet records */
int ids_compare_genid(const alert_t *soc_a, const alert_t *soc_b);
int ids_match_destip(const alert_t *rec_a, const alert_t *rec_b);

/* functions to create and cleanup alert lists */
llist_t *ids_create(const char *);
void ids_cleanup(llist_t *);

/* Functions to get and print packet information */
void ids_print(llist_t *list_ptr, const char *); /* print list of records */
void ids_stats(llist_t *, llist_t *);           /* prints size of each list */

/* functions for sorted list */
void ids_add(llist_t *);
void ids_list_gen(llist_t *, int);
void ids_list_ip(llist_t *, int);
void ids_remove_gen(llist_t *, int);
void ids_remove_ip(llist_t *, int);
void ids_scan(llist_t *, int);

/* functions for unsorted list
 * inserts at the rear, removes at the front,
 * Duplicate packets with the same destination IP address are not
 * permitted. If a new packet is inserted the old one must be
 * recycled
 */
void ids_add_rear(llist_t *);
void ids_remove_front(llist_t *);
```

Extended user interface for managing packet records

Implement the user functions from MP1 that control a sorted list. These commands should produce identical results compared to MP1. Unlike with MP1, there is no need to specify the size of the initial list. So you *must not* read a command line argument when the MP2 program is run.

```
ALERT
LISTGEN gen-id
LISTIP ip-address
DELGEN gen-id
DELIP ip-address
SCANGEN threshold
PRINT
QUIT
```

In addition add the following four user functions for a *second* list that is *unsorted* and does *not* have a limit on the number of elements in the list. However, there can never be more than one packet from a specific IP address in the list. So, the ADDREAR command first checks if a packet with the same destination ip_address is already in the list. If so, the packet is removed and freed. In any case, the new packet is appended to the end of the list (the new packet is **not** inserted into the position of the old packet). The RMFRONT command simply removes the record at the front of the list, if any.

```
ADDREAR
RMFRONT
PRINTQ
```

STATS prints the size of both the sorted and unsorted lists
QUIT ends the program and cleans up both lists.

Your program maintains two lists, one sorted and one unsorted. The first group of commands operate on the sorted list. The second three commands operate on the unsorted list.

To facilitate grading, the output for each command must be formatted exactly as provided in the supplemental template programs. The `printf()` commands in `ids_support.c` must not be changed.

Submit one ZIP file to Canvas containing `lab2.c`, `ids_support.c`, `l1ist.c`, `datatypes.h`, `ids_support.h`, `l1ist.h`, `makefile`, and one input test file you created. Do not include `driver.c`, executables, object files, or provided tests. See `README.md` for details. Multiple submissions are allowed; only the latest submission before the Canvas deadline will be graded.

To receive credit for this assignment your code must compile and at a minimum evaluate the provided `testinput` and identically match the `expectedoutput`. Code that does not compile or crashes before completing the example input will not be accepted or graded. If your code is not a perfect match you must contact the instructor or TA and fix your code. Code that correctly handles the example input but otherwise fails many other cases will be scored at 50% or less.

See the ECE 2230 Programming Guide for additional requirements that apply to all programming assignments.

Work must be completed by each individual student, and see the course syllabus for additional policies. The work you submit should be your own words and computer code that you wrote for this assignment.